

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester	V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY	2026-27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n) \quad (1)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1 \quad (2)$$

where

N Input Size
 F Filter Size
 P Padding
 S Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications: Object detection, Lane detection, Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications: Text detection, Boundary extraction, Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications: Edge enhancement, Satellite image analysis, Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications: Image enhancement, Optical Character Recognition (OCR), Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications: Noise removal, Image smoothing, Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications: Noise reduction, Computer vision preprocessing, Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications: Texture analysis, Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications: Image segmentation, Boundary detection, Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications: Motion simulation, Image restoration, Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4 Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position: $1(1) + 2(0) + 4(0) + 5(1) = 6$

Second position: $2(1) + 3(0) + 5(0) + 6(1) = 8$

Third position: $4(1) + 5(0) + 7(0) + 8(1) = 12$

Fourth position: $5(1) + 6(0) + 8(0) + 9(1) = 14$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5 Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

$$\text{Window 1: } \begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

$$\text{Window 2: } \begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

$$\text{Window 3: } \begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

$$\text{Window 4: } \begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6 Numerical Example 3: Parameter Calculation

Suppose

Input Image: $32 \times 32 \times 3$

Convolution Layer:

- Filters = 16
- Kernel = 3×3

Parameters

$$\begin{aligned} & (3 \times 3 \times 3 + 1) \times 16 \\ &= (27 + 1) \times 16 \\ &= 448 \end{aligned}$$

Therefore, Total trainable parameters = 448.

7 Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8 Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Observed Output:

Train set shape: (50000, 32, 32, 3), Labels shape: (50000, 1)
 Test set shape: (10000, 32, 32, 3), Labels shape: (10000, 1)

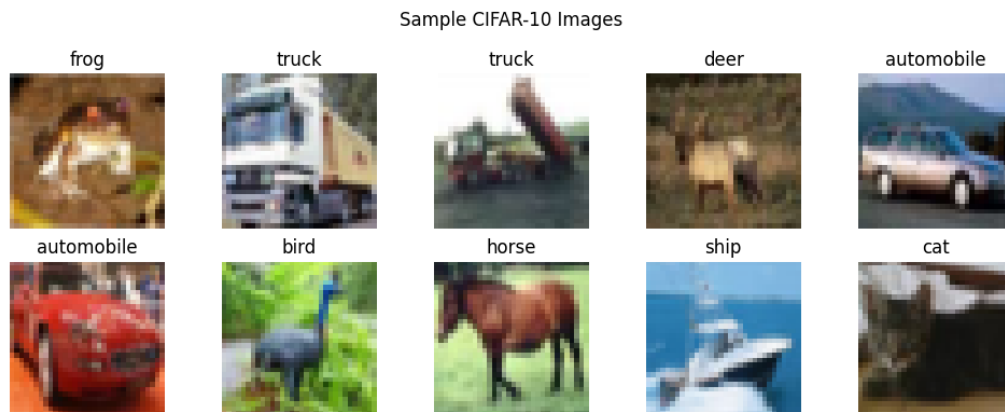


Figure 1: Ten sample images from the CIFAR-10 training set (Plot 1).

Inference: The images are 32×32 pixel color images belonging to 10 balanced visual object classes.

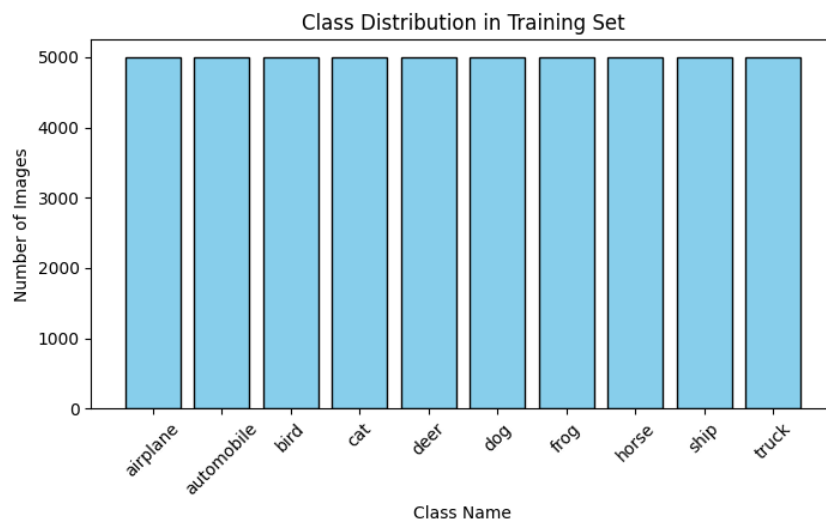


Figure 2: Class distribution in the training set (Plot 2).

Inference: The training set is perfectly balanced with 5,000 samples per class across all 10 categories.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Observed Output (Kernel Size, padding = valid, input $32 \times 32 \times 3$):

Kernel	Padding	Output Shape
3×3	valid	30×30
5×5	valid	28×28
7×7	valid	26×26

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Observed Output (Kernel 3×3 , input $32 \times 32 \times 3$):

Stride	Padding	Output Shape
1	valid	30×30
1	same	32×32
2	valid	15×15
2	same	16×16

Larger kernels shrink the output faster under valid padding where as same padding keeps the spatial size fixed at stride 1 and roughly halves it at stride 2.

Task 4

Visualize feature maps after the first convolution layer. Display at least eight feature maps. (See Figure 6 in the Mandatory Plots section.)

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Observed Output:

MaxPooling2D output shape: (16, 16)

AveragePooling2D output shape: (16, 16)

Max Pooling final val accuracy: 0.6826

Average Pooling final val accuracy: 0.6771

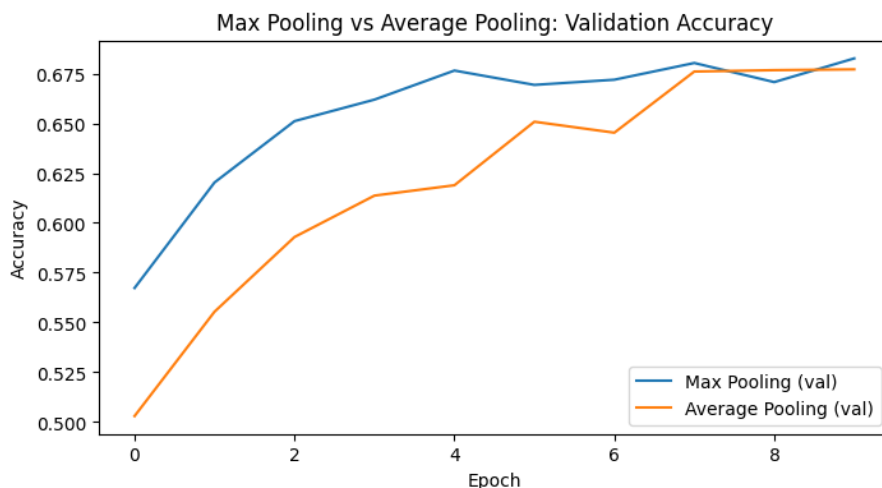


Figure 3: Validation accuracy: Max Pooling vs. Average Pooling (10 epochs).

Inference: Both the pooling types reduce the spatial size identically (16×16), since they differ only in how each 2×2 window is summarized. Max pooling reaches a slightly higher validation accuracy of 68.26% than average pooling which is 67.71%. This is consistent with max pooling preserving the strongest activations.

Task 6

Construct the following CNN.

Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Model Summary:

Layer (type)	Output Shape	Param #
Conv2D	(None, 32, 32, 16)	448
MaxPooling2D	(None, 16, 16, 16)	0
Conv2D	(None, 16, 16, 32)	4,640
MaxPooling2D	(None, 8, 8, 32)	0
Flatten	(None, 2048)	0
Dense	(None, 64)	131,136
Dense	(None, 10)	650

Total params: 136,874 (534.66 KB) — all trainable.

Training Log (selected epochs, Adam, batch size 32):

Epoch	Train Acc	Train Loss	Val Acc	Val Loss
1	0.4579	1.5064	0.5627	1.2416
5	0.6878	0.8941	0.6645	0.9733
10	0.7480	0.7171	0.6684	0.9821
15	0.7907	0.5962	0.6731	1.0388
20	0.8209	0.5007	0.6739	1.1839

Inference: Training accuracy climbs steadily to 82.09% by epoch 20. Validation accuracy plateaus around 67% from roughly epoch 8 onward and validation loss starts rising after epoch 7. This is a very clear sign of overfitting in the later iterations.

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Confusion matrix: see Figure 7 in the Mandatory Plots section.

Classification Report:

Class	Precision	Recall	F1-score	Support
airplane	0.64	0.78	0.70	1000
automobile	0.75	0.82	0.78	1000
bird	0.61	0.52	0.56	1000
cat	0.55	0.39	0.45	1000
deer	0.60	0.64	0.62	1000
dog	0.56	0.64	0.60	1000
frog	0.77	0.74	0.75	1000
horse	0.70	0.76	0.73	1000
ship	0.85	0.71	0.77	1000
truck	0.72	0.75	0.73	1000
accuracy		0.67		10000
macro avg	0.67	0.67	0.67	10000
weighted avg	0.67	0.67	0.67	10000

Inference: The model performs best on structurally distinct classes (automobile, ship, frog) and worst on cat - i.e. showing that it does not perform as well on classes that look similar.

9 Mandatory Plots

The eight mandatory plots required by Section 9 are: Sample Images and Class Distribution (embedded under Task 1), Training/Validation Accuracy, Training/Validation Loss, Feature Maps, and the Confusion Matrix, embedded below.

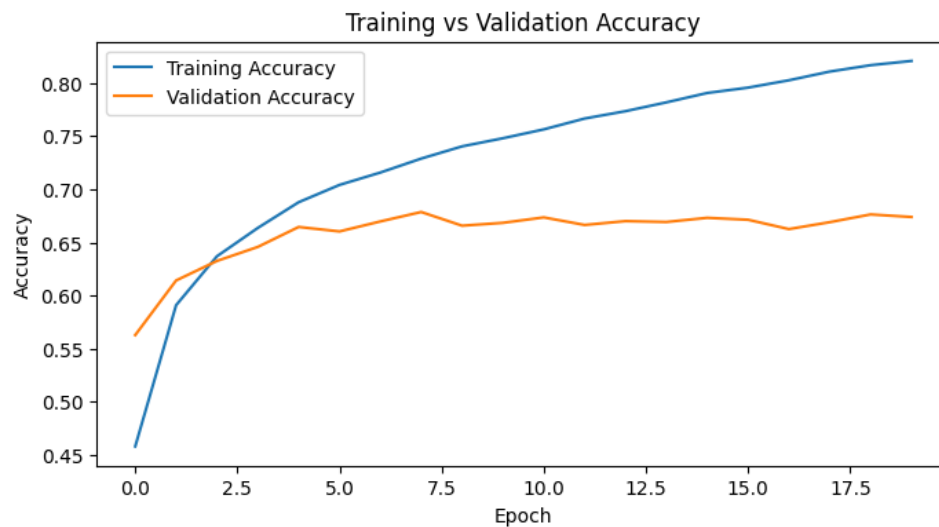


Figure 4: Training vs. Validation Accuracy (Plots 3 & 4).

Inference: Accuracy improves steadily, but the gap widens between training and validation accuracy in the later epochs. This is a clear sign of overfitting.

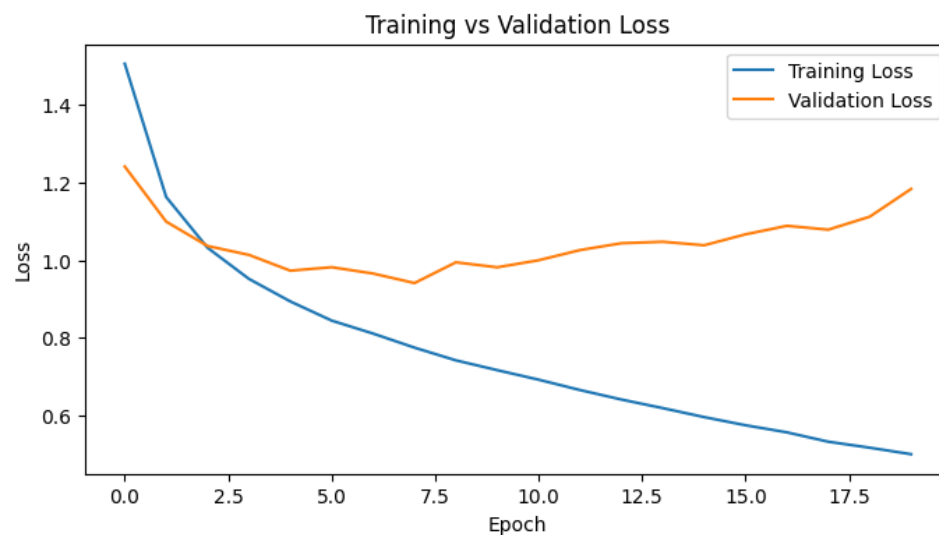


Figure 5: Training vs. Validation Loss (Plots 5 & 6).

Inference: Training loss decreases continuously. Validation loss begins increasing very slowly from around epoch 7, signalling that it would have been better to stop a bit earlier.

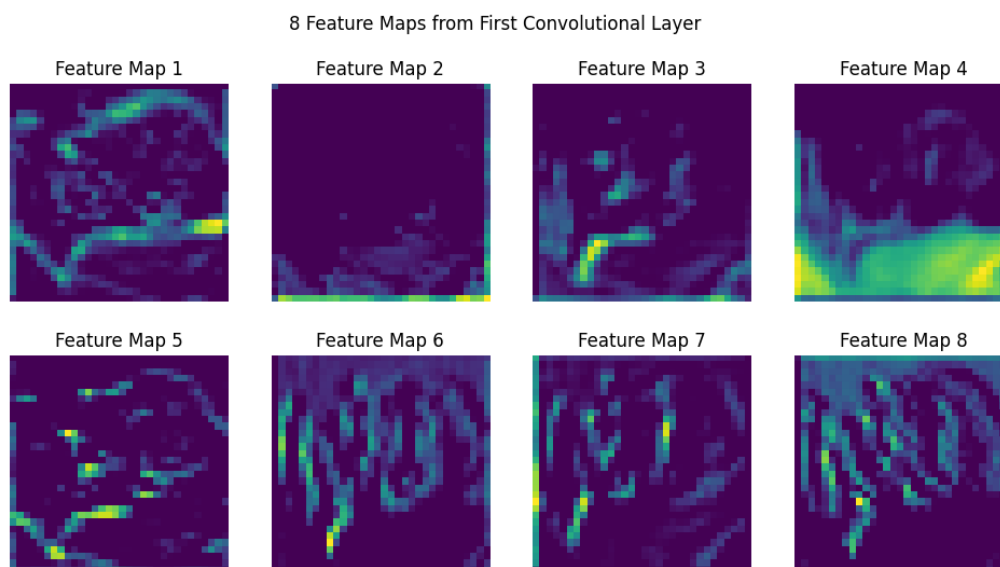


Figure 6: Eight feature maps from the first convolutional layer (Plot 7).

Inference: The low level convolutional filters extract basic visual features like edges, textures, and color gradients.

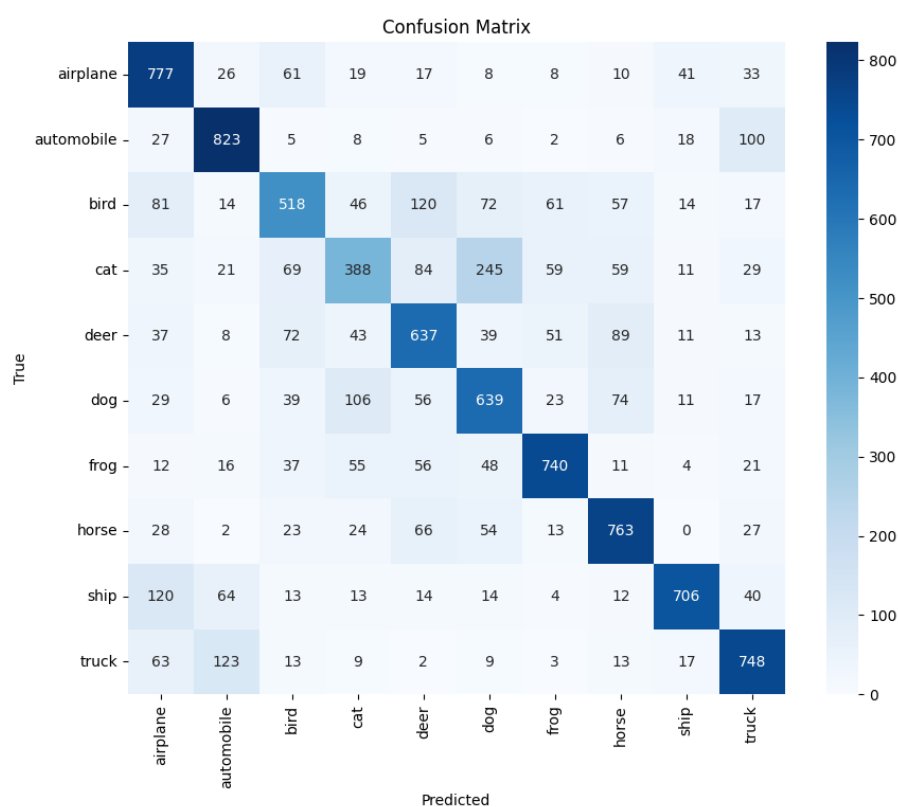


Figure 7: Confusion Matrix on the CIFAR-10 test set (Plot 8).

Inference: High diagonal values show correct classifications, while confusion mainly occurs between visually similar classes like cats and dogs.

10 Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Observed Output:

Formula output size: $32.5 \rightarrow \text{floor} = 32$

Verified via Keras layer: output shape = (32, 32)

Applying $\frac{N - F + 2P}{S} + 1 = \frac{64 - 5 + 4}{2} + 1 = 32.5$, which Keras rounds down to an output of 32×32 .

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

Observed Output:

Trainable parameters: $(3 \times 3 \times 3 + 1) \times 64 = 1792$

Verified via Keras layer: 1792 parameters

3. Compare ReLU and Sigmoid activation functions.

Observed Output (10 epochs each, otherwise identical architecture):

ReLU final val accuracy: 0.6871

Sigmoid final val accuracy: 0.5592

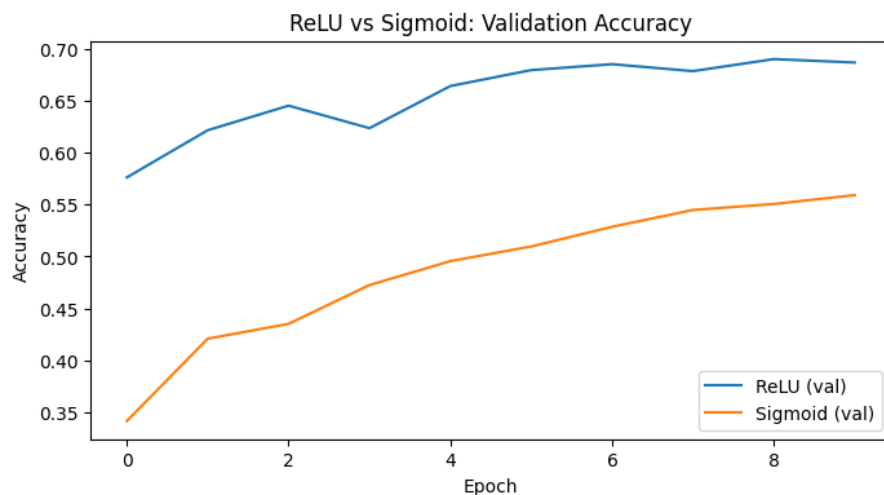


Figure 8: Validation accuracy: ReLU vs. Sigmoid activation (10 epochs).

Inference: ReLU converges faster and reaches a substantially higher validation accuracy (68.71%) than Sigmoid (55.92%). This is because Sigmoid saturates when given inputs of large magnitude and suffers from vanishing gradients. This slows learning.

4. Replace Max Pooling with Average Pooling and compare the results.

See the Task 5 comparison above: Max Pooling (68.26% val accuracy) slightly outperforms Average Pooling (67.71% val accuracy) with identical output sizes.

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

Accuracy will almost certainly increase. However the increase in computation time makes the tradeoff debateable.

11 Results

Metric	Value
Training Accuracy (epoch 20)	82.09%
Testing / Validation Accuracy	67.39% (final epoch); 67% overall (test set)
Precision (weighted avg)	0.67
Recall (weighted avg)	0.67
F1-score (weighted avg)	0.67
Number of Parameters	136,874 (all trainable)

12 Discussion

Answer the following.

1. **Why is convolution preferred over fully connected layers for images?**

Convolution is preferred because it makes use of the spatial structure of images. Convolutional filters operate on small portions and can detect features such as edges, textures, and patterns. They also use shared weights, which reduces the training required.

2. **How does stride affect the feature map size?**

Stride determines how many pixels the convolution filter moves at each step. A larger stride produces a smaller feature map, while a smaller stride produces a larger feature map. The output size is given by

$$\frac{N - F + 2P}{S} + 1$$

where N is the input size, F is the filter size, P is the padding, and S is the stride.

3. **What is the role of padding?**

Padding adds extra pixels around the border of the input image. It helps preserve the spatial dimensions of the image and allows the convolution filter to process pixels near the edges properly.

4. **Why is pooling used?**

Pooling is used to reduce the spatial dimensions of feature maps. This reduces computation time while retaining important features. Common pooling methods are max pooling and average pooling.

5. **How do feature maps represent image characteristics?**

Feature maps represent the features detected by convolutional filters. Different filters can detect characteristics such as edges, textures, corners, and other visual patterns. The feature map is used to indicate where these characteristics occur in the image.

6. **Why do CNNs require fewer parameters than MLPs?**

CNNs require fewer parameters because they use local connectivity and weight sharing. The same convolutional filter is applied across different portions of an image instead of using separate weights for every input pixel and neuron and redoing it.

13 References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.